

CMU 14-455 (2019) · 课程资料包 @ShowMeAI



视频

中英双语字幕



课件

一键打包下载



笔记

官方笔记翻译



代码

作业项目解析



视频 · B 站 [扫码或点击链接]

<https://www.bilibili.com/video/BV1qf4y1J7mX>



课件 & 代码 · 博客 [扫码或点击链接]

<http://blog.showmeai.tech/cmu-14-455/>

高级 SQL
数据库

聚合
内存管理
树索引

排序
查询规划
索引构建

并发控制
查询执行

Awesome AI Courses Notes Cheatsheets 是 [ShowMeAI](#) 资料库的分支系列，覆盖最具知名度的 **TOP50+** 门 AI 课程，旨在为读者和学习者提供一整套高品质中文学习笔记和速查表。

点击课程名称，跳转至课程**资料包**页面，**一键下载**课程全部资料！

机器学习	深度学习	自然语言处理	计算机视觉
Stanford · CS229	Stanford · CS230	Stanford · CS224n	Stanford · CS231n
# Awesome AI Courses Notes Cheatsheets · 持续更新中			
知识图谱	图机器学习	深度强化学习	自动驾驶
Stanford · CS520	Stanford · CS224W	UCBerkeley · CS285	MIT · 6.S094



微信公众号

资料下载方式 2：扫码点击**底部菜单栏**
称为 **AI 内容创作者**？回复 [添砖加瓦]

12

Query Execution – Part I



Intro to Database Systems
15-445/15-645
Fall 2019

AP

Andy Pavlo
Computer Science
Carnegie Mellon University

ADMINISTRIVIA

Homework #3 is due Wed Oct 9th @ 11:59pm

Mid-Term Exam is Wed Oct 16th @ 12:00pm

Project #2 is due Sun Oct 20th @ 11:59pm

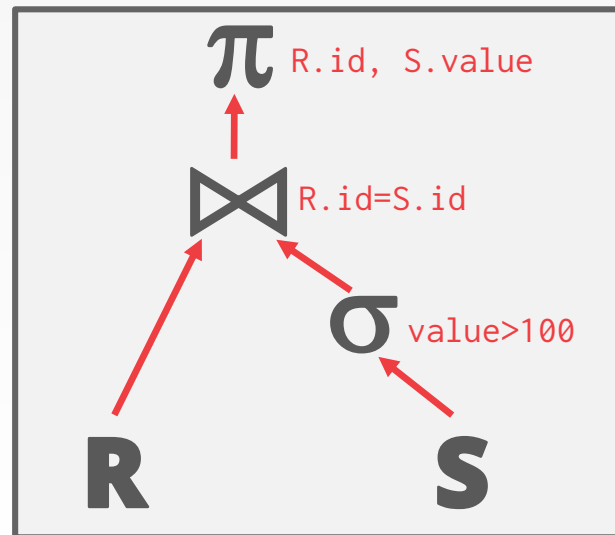
QUERY PLAN

The operators are arranged in a tree.

Data flows from the leaves of the tree up towards the root.

The output of the root node is the result of the query.

```
SELECT R.id, S.cdate  
FROM R JOIN S  
      ON R.id = S.id  
WHERE S.value > 100
```



TODAY'S AGENDA

Processing Models

Access Methods

Expression Evaluation



PROCESSING MODEL

A DBMS's **processing model** defines how the system executes a query plan.

→ Different trade-offs for different workloads.

Approach #1: Iterator Model

Approach #2: Materialization Model

Approach #3: Vectorized / Batch Model

ITERATOR MODEL

Each query plan operator implements a **Next** function.

- On each invocation, the operator returns either a single tuple or a null marker if there are no more tuples.
- The operator implements a loop that calls next on its children to retrieve their tuples and then process them.

Also called **Volcano** or **Pipeline** Model.

ITERATOR MODEL

Next()

```
for t in child.Next():
    emit(projection(t))
```

Next()

```
for t1 in left.Next():
    buildHashTable(t1)
for t2 in right.Next():
    if probe(t2): emit(t1 ⋈ t2)
```

Next()

```
for t in child.Next():
    if evalPred(t): emit(t)
```

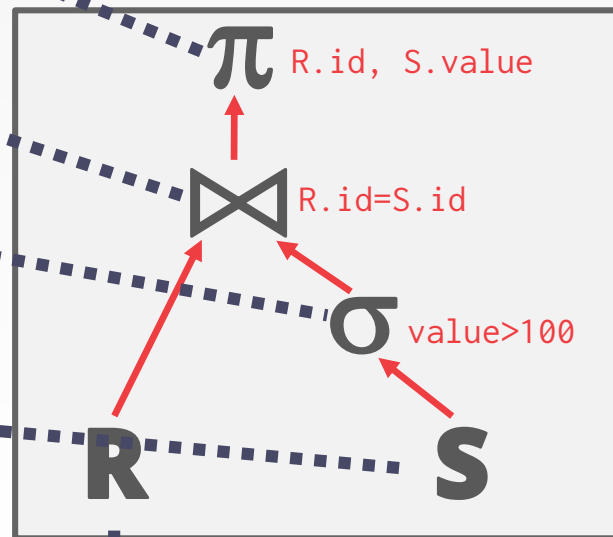
Next()

```
for t in R:
    emit(t)
```

Next()

```
for t in S:
    emit(t)
```

```
SELECT R.id, S.cdate
FROM R JOIN S
ON R.id = S.id
WHERE S.value > 100
```



ITERATOR MODEL

1

```
for t in child.Next():
    emit(projection(t))
```

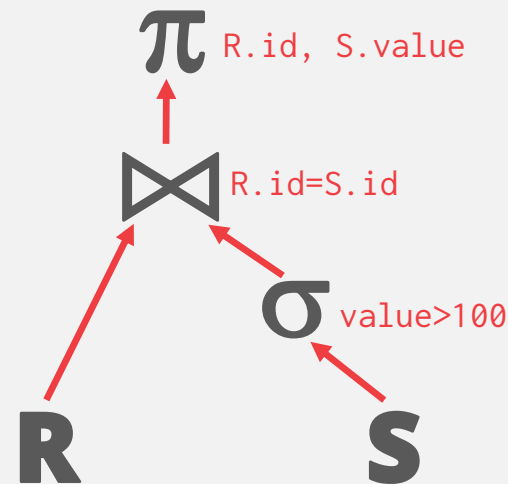
```
for t1 in left.Next():
    buildHashTable(t1)
for t2 in right.Next():
    if probe(t2): emit(t1 ⋈ t2)
```

```
for t in child.Next():
    if evalPred(t): emit(t)
```

```
for t in R:
    emit(t)
```

```
for t in S:
    emit(t)
```

```
SELECT R.id, S.cdate
FROM R JOIN S
      ON R.id = S.id
WHERE S.value > 100
```



ITERATOR MODEL

1

```
for t in child.Next():
    emit(projection(t))
```

2

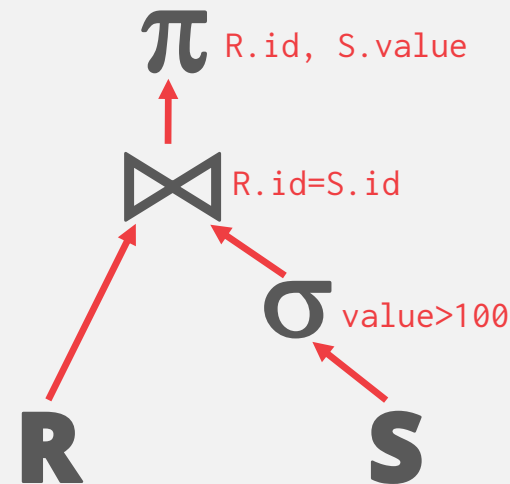
```
for t1 in left.Next():
    buildHashTable(t1)
for t2 in right.Next():
    if probe(t2): emit(t1 ⋈ t2)
```

```
for t in child.Next():
    if evalPred(t): emit(t)
```

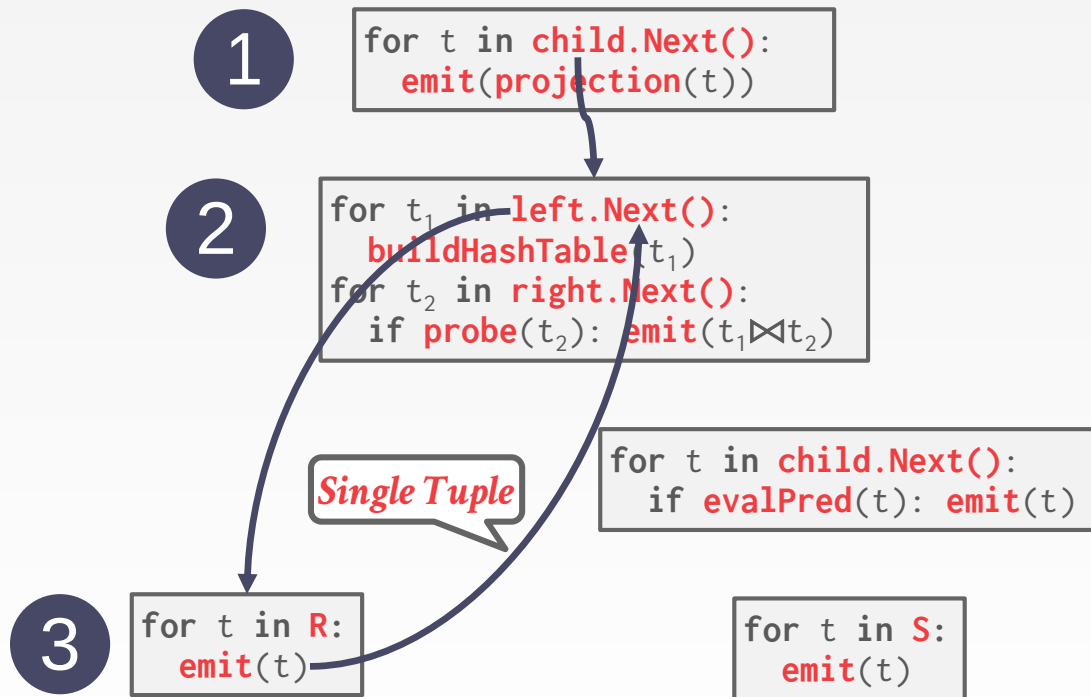
```
for t in R:
    emit(t)
```

```
for t in S:
    emit(t)
```

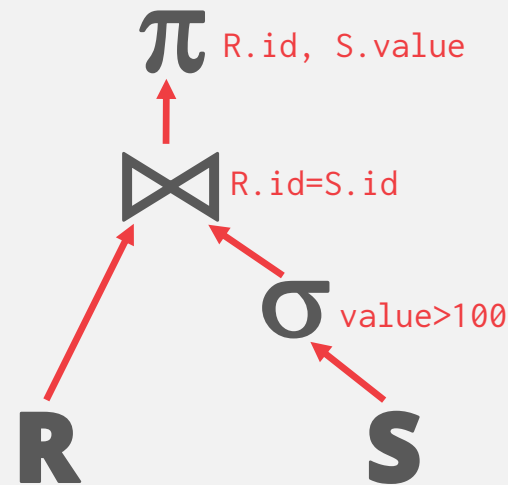
```
SELECT R.id, S.cdate
FROM R JOIN S
      ON R.id = S.id
WHERE S.value > 100
```



ITERATOR MODEL



```
SELECT R.id, S.cdate
FROM R JOIN S
      ON R.id = S.id
WHERE S.value > 100
```



ITERATOR MODEL

1

```
for t in child.Next():
    emit(projection(t))
```

2

```
for t1 in left.Next():
    buildHashTable(t1)
for t2 in right.Next():
    if probe(t2): emit(t1 ⋈ t2)
```

```
for t in child.Next():
    if evalPred(t): emit(t)
```

4

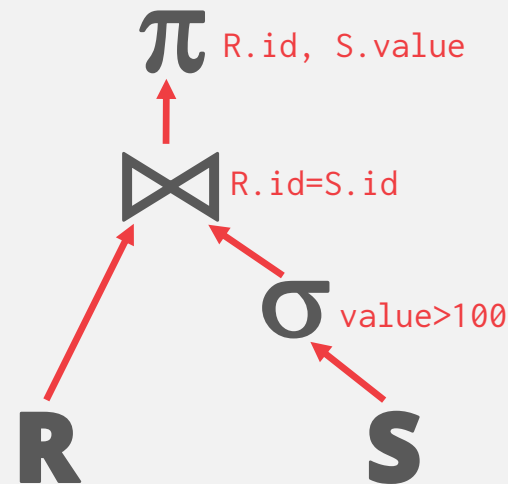
3

```
for t in R:
    emit(t)
```

5

```
for t in S:
    emit(t)
```

```
SELECT R.id, S.cdate
FROM R JOIN S
      ON R.id = S.id
WHERE S.value > 100
```



ITERATOR MODEL

1

```
for t in child.Next():
    emit(projection(t))
```

2

```
for t1 in left.Next():
    buildHashTable(t1)
for t2 in right.Next():
    if probe(t2): emit(t1 ⋈ t2)
```

```
for t in child.Next():
    if evalPred(t): emit(t)
```

4

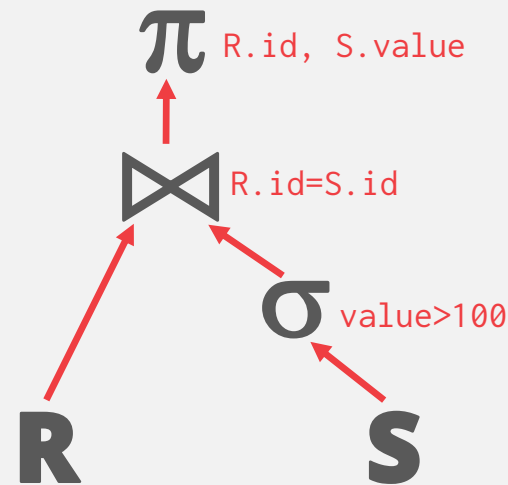
3

```
for t in R:
    emit(t)
```

5

```
for t in S:
    emit(t)
```

```
SELECT R.id, S.cdate
FROM R JOIN S
      ON R.id = S.id
WHERE S.value > 100
```



ITERATOR MODEL

This is used in almost every DBMS. Allows for tuple **pipelining**.

Some operators have to block until their children emit all of their tuples.

→ Joins, Subqueries, Order By

Output control works easily with this approach.



MATERIALIZATION MODEL

Each operator processes its input all at once and then emits its output all at once.

- The operator "materializes" its output as a single result.
- The DBMS can push down hints into to avoid scanning too many tuples.
- Can send either a materialized row or a single column.

The output can be either whole tuples (NSM) or subsets of columns (DSM)

MATERIALIZATION MODEL

1

```

out = [ ]
for t in child.Output():
    out.add(projection(t))
return out

```

```

out = [ ]
for t1 in left.Output():
    buildHashTable(t1)
for t2 in right.Output():
    if probe(t2): out.add(t1 ⋈ t2)
return out

```

```

out = [ ]
for t in child.Output():
    if evalPred(t): out.add(t)
return out

```

```

out = [ ]
for t in R:
    out.add(t)
return out

```

```

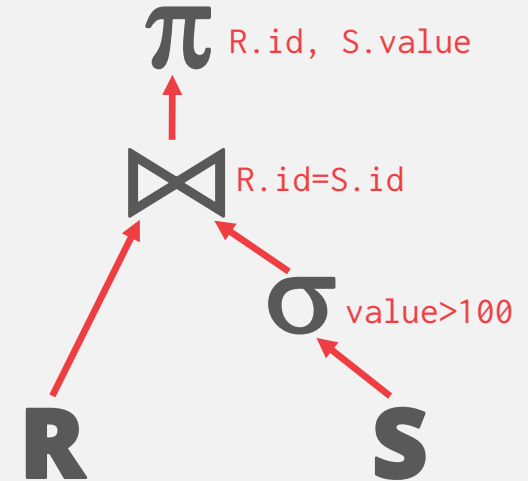
out = [ ]
for t in S:
    out.add(t)
return out

```

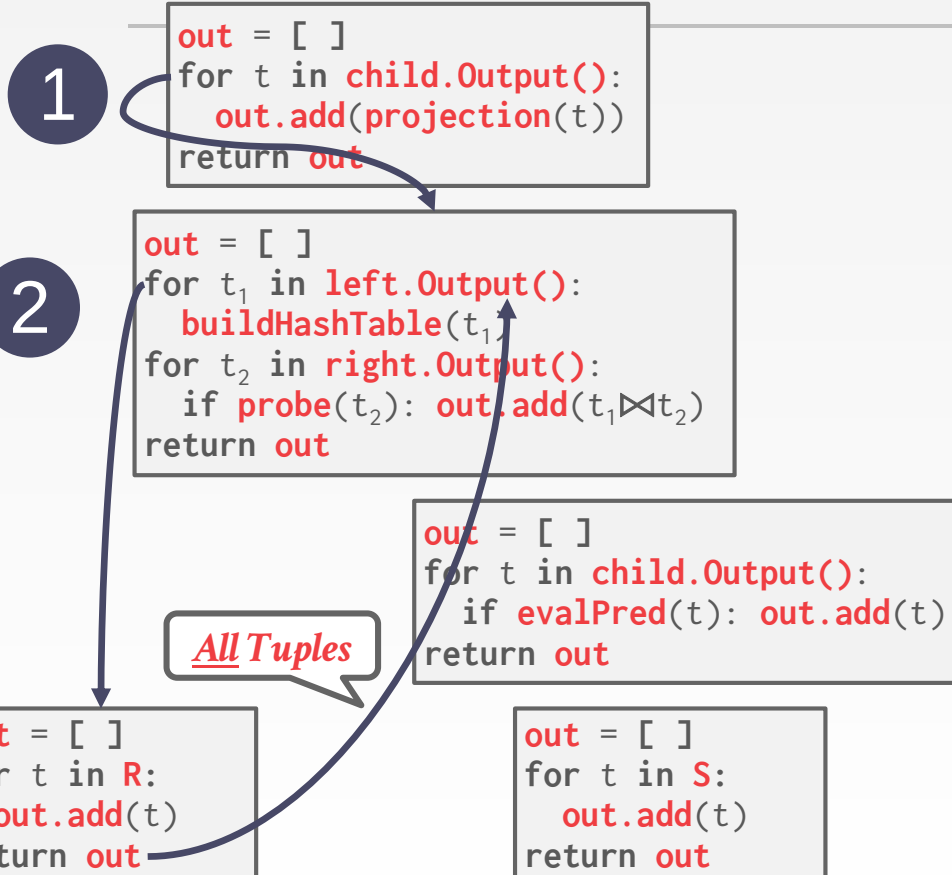
```

SELECT R.id, S.cdate
FROM R JOIN S
      ON R.id = S.id
WHERE S.value > 100

```



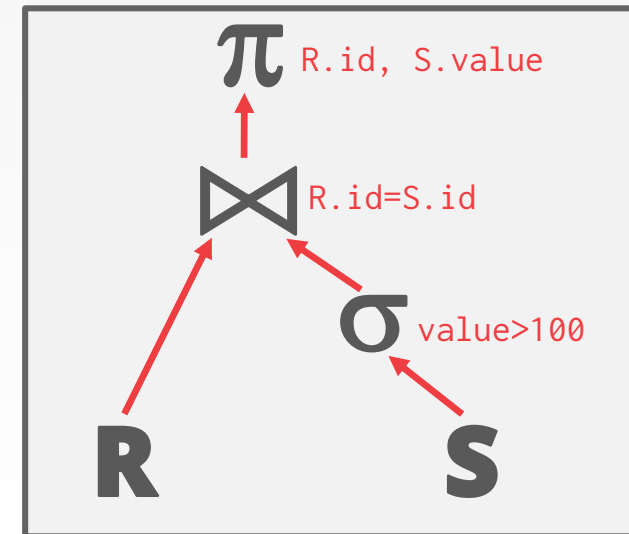
MATERIALIZATION MODEL



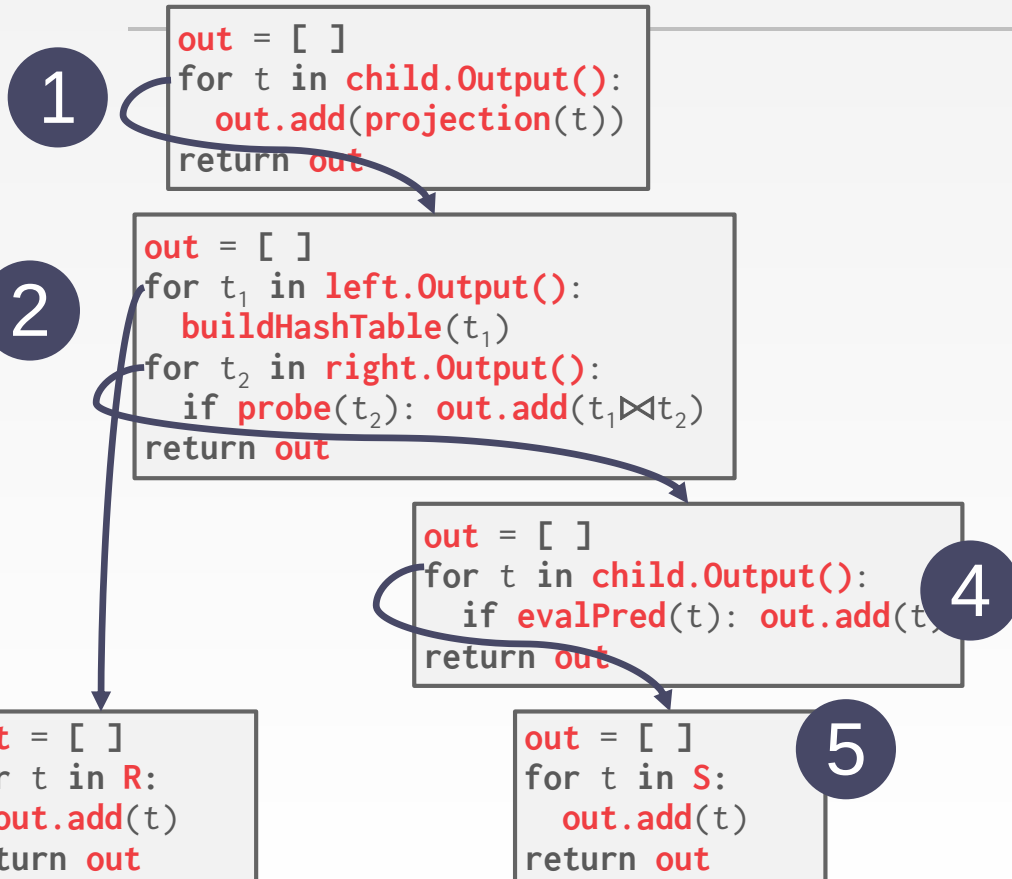
```

SELECT R.id, S.cdate
FROM R JOIN S
      ON R.id = S.id
WHERE S.value > 100

```

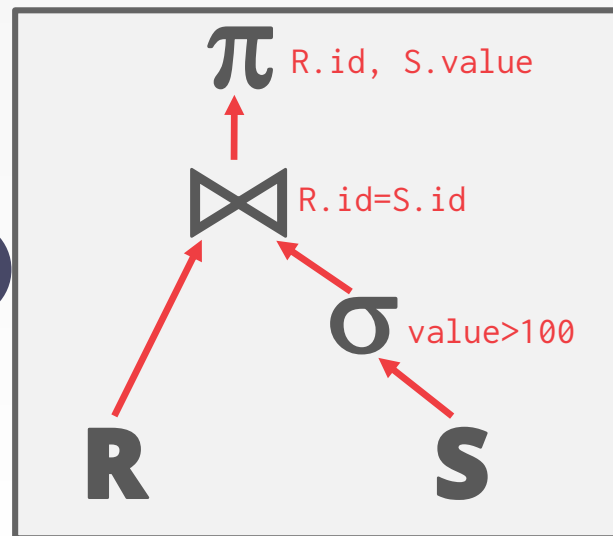


MATERIALIZATION MODEL

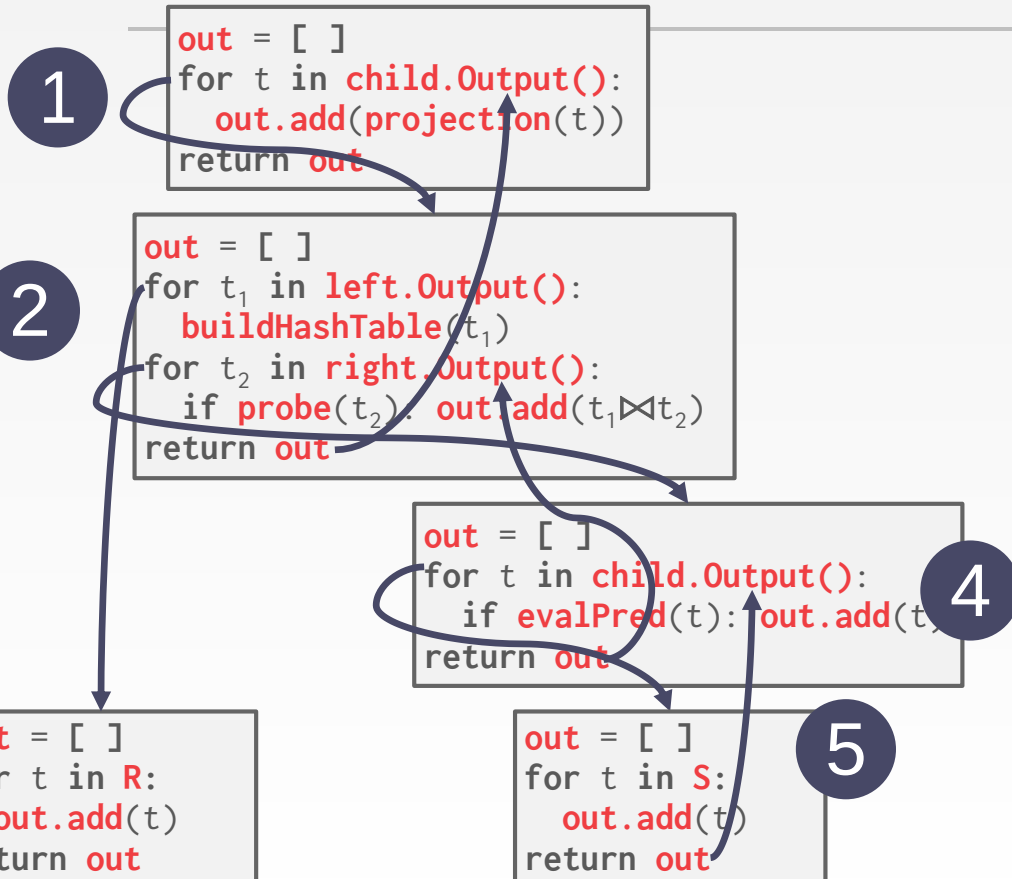


```

SELECT R.id, S.cdate
FROM R JOIN S
      ON R.id = S.id
WHERE S.value > 100
  
```



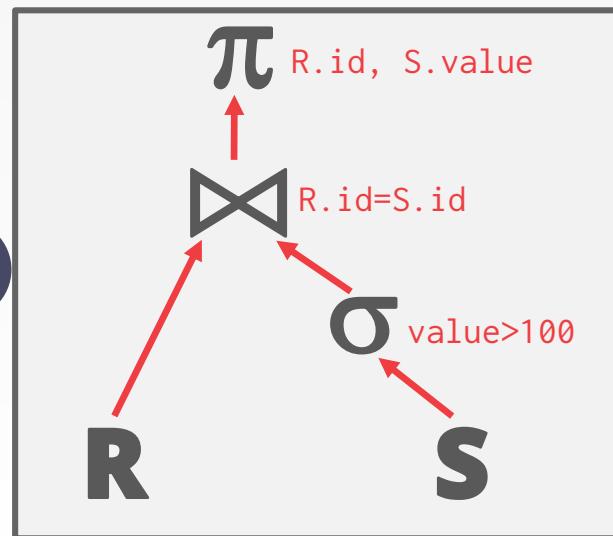
MATERIALIZATION MODEL



```

SELECT R.id, S.cdate
FROM R JOIN S
      ON R.id = S.id
WHERE S.value > 100

```



MATERIALIZATION MODEL

Better for OLTP workloads because queries only access a small number of tuples at a time.

- Lower execution / coordination overhead.
- Fewer function calls.

Not good for OLAP queries with large intermediate results.



VECTORIZATION MODEL

Like the Iterator Model where each operator implements a **Next** function in this model.

Each operator emits a **batch** of tuples instead of a single tuple.

- The operator's internal loop processes multiple tuples at a time.
- The size of the batch can vary based on hardware or query properties.

VECTORIZATION MODEL

1
`out = [ ]`
`for t in child.Next():`
`out.add(projection(t))`
`if |out|>n: emit(out)`

2
`out = [ ]`
`for t1 in left.Next():`
`buildHashTable(t1)`
`for t2 in right.Next():`
`if probe(t2): out.add(t1 ⋈ t2)`
`if |out|>n: emit(out)`

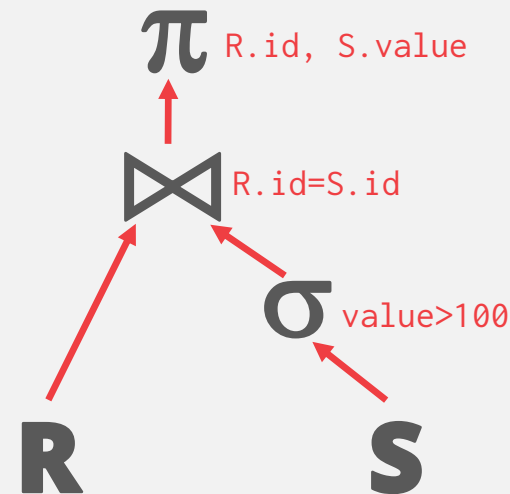
`out = [ ]`
`for t in child.Next():`
`if evalPred(t): out.add(t)`
`if |out|>n: emit(out)`

3
`out = [ ]`
`for t in R:`
`out.add(t)`
`if |out|>n: emit(out)`

Tuple Batch

`out = [ ]`
`for t in S:`
`out.add(t)`
`if |out|>n: emit(out)`

SELECT R.id, S.cdate
 FROM R JOIN S
 ON R.id = S.id
 WHERE S.value > 100



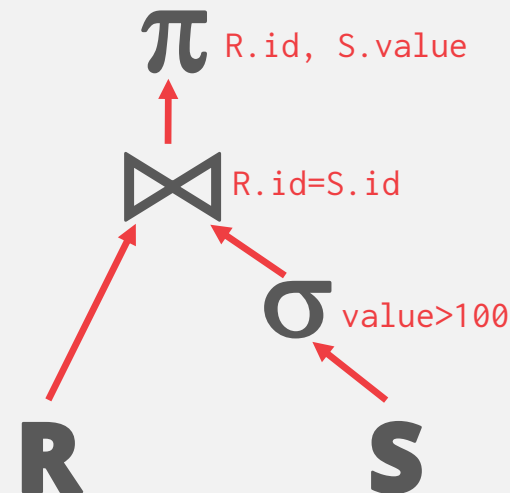
VECTORIZATION MODEL

1
`out = [ ]`
`for t in child.Next():`
`out.add(projection(t))`
`if |out|>n: emit(out)`

2
`out = [ ]`
`for t1 in left.Next():`
`buildHashTable(t1)`
`for t2 in right.Next():`
`if probe(t2): out.add(t1 <math>\bowtie</math> t2)`
`if |out|>n: emit(out)`

4
`out = [ ]`
`for t in child.Next():`
`if evalPred(t): out.add(t)`
`if |out|>n: emit(out)`

SELECT R.id, S.cdate
 FROM R JOIN S
 ON R.id = S.id
 WHERE S.value > 100



3
`out = [ ]`
`for t in R:`
`out.add(t)`
`if |out|>n: emit(out)`

Tuple Batch

5
`out = [ ]`
`for t in S:`
`out.add(t)`
`if |out|>n: emit(out)`

VECTORIZATION MODEL

Ideal for OLAP queries because it greatly reduces the number of invocations per operator.

Allows for operators to use vectorized (SIMD) instructions to process batches of tuples.



PLAN PROCESSING DIRECTION

Approach #1: Top-to-Bottom

- Start with the root and "pull" data up from its children.
- Tuples are always passed with function calls.

Approach #2: Bottom-to-Top

- Start with leaf nodes and push data to their parents.
- Allows for tighter control of caches/registers in pipelines.

ACCESS METHODS

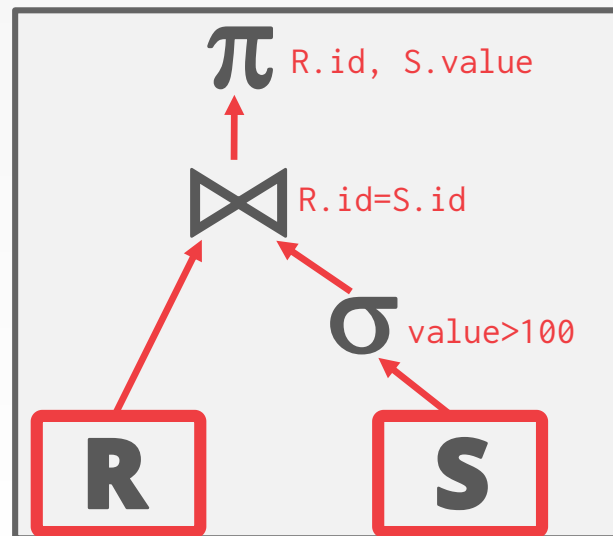
An **access method** is a way that the DBMS can access the data stored in a table.

→ Not defined in relational algebra.

Three basic approaches:

- Sequential Scan
- Index Scan
- Multi-Index / "Bitmap" Scan

```
SELECT R.id, S.cdate  
FROM R JOIN S  
      ON R.id = S.id  
WHERE S.value > 100
```



SEQUENTIAL SCAN

For each page in the table:

- Retrieve it from the buffer pool.
- Iterate over each tuple and check whether to include it.

The DBMS maintains an internal **cursor** that tracks the last page / slot it examined.

```
for page in table.pages:  
    for t in page.tuples:  
        if evalPred(t):  
            // Do Something!
```

SEQUENTIAL SCAN: OPTIMIZATIONS

This is almost always the worst thing that the DBMS can do to execute a query.

Sequential Scan Optimizations:

- Prefetching
- Buffer Pool Bypass
- Parallelization
- Zone Maps
- Late Materialization
- Heap Clustering





ZONE MAPS



Pre-computed aggregates for the attribute values in a page. DBMS checks the zone map first to decide whether it wants to access the page.

```
SELECT * FROM table
WHERE val > 600
```

Original Data

val
100
200
300
400
400

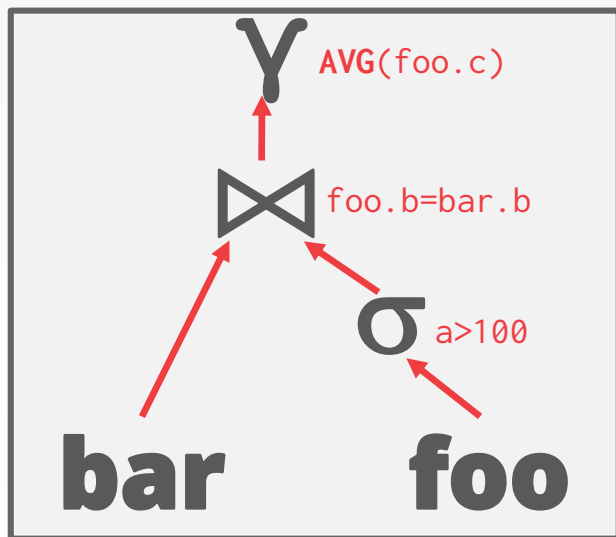


Zone Map

type	val
MIN	100
MAX	400
AVG	280
SUM	1400
COUNT	5

LATE MATERIALIZATION

DSM DBMSs can delay stitching together tuples until the upper parts of the query plan.

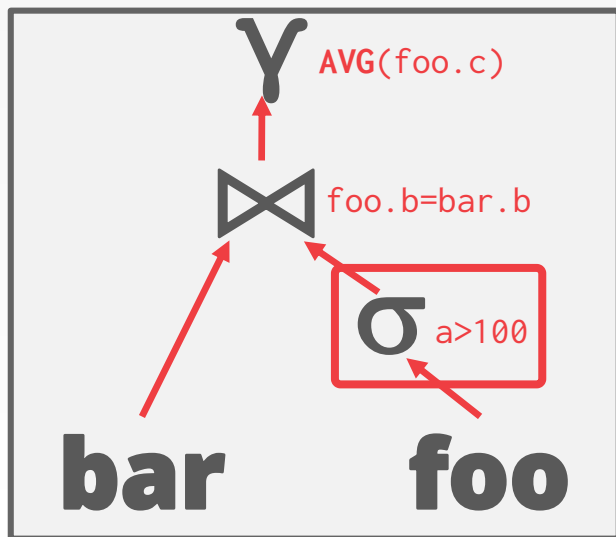


```
SELECT AVG(foo.c)
FROM foo JOIN bar
ON foo.b = bar.b
WHERE foo.a > 100
```

	a	b	c
0			
1			
2			
3			

LATE MATERIALIZATION

DSM DBMSs can delay stitching together tuples until the upper parts of the query plan.



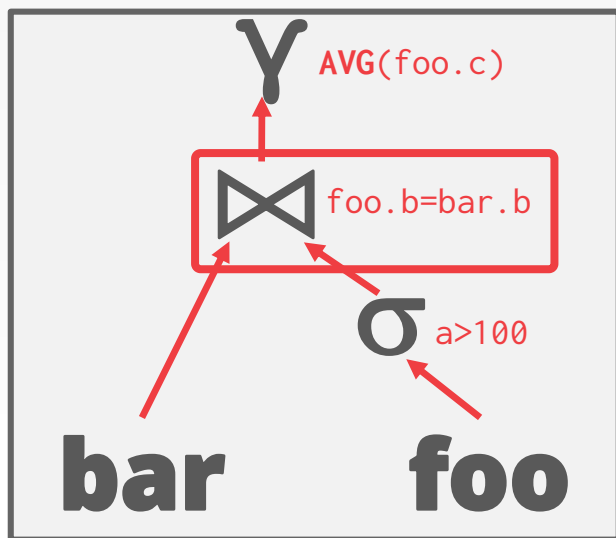
Offsets

```
SELECT AVG(foo.c)
FROM foo JOIN bar
ON foo.b = bar.b
WHERE foo.a > 100
```

	a	b	c
0			
1			
2			
3			

LATE MATERIALIZATION

DSM DBMSs can delay stitching together tuples until the upper parts of the query plan.

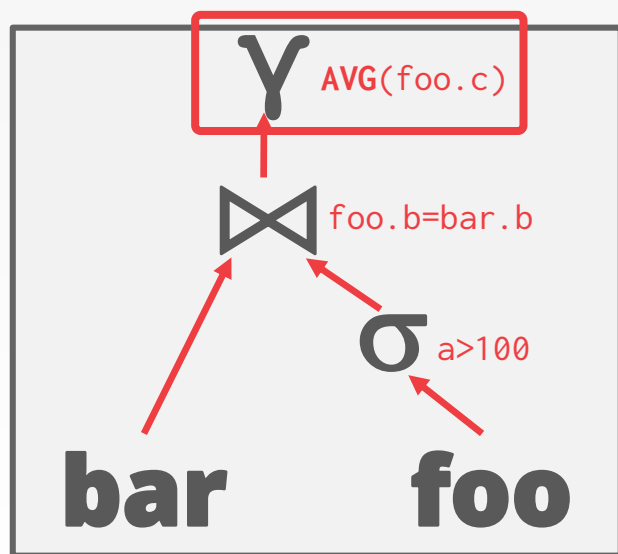


```
SELECT AVG(foo.c)
FROM foo JOIN bar
ON foo.b = bar.b
WHERE foo.a > 100
```

	a	b	c
0			
1			
2			
3			

LATE MATERIALIZATION

DSM DBMSs can delay stitching together tuples until the upper parts of the query plan.



↑
Result
↑
Offsets
↑
Offsets

```

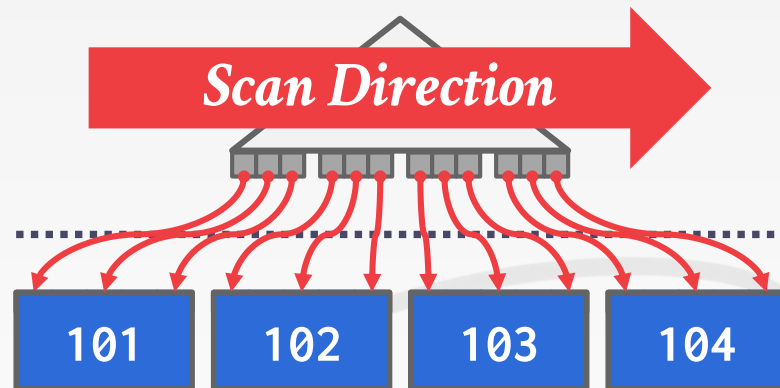
SELECT AVG(foo.c)
FROM foo JOIN bar
ON foo.b = bar.b
WHERE foo.a > 100
  
```

	a	b	c
0			
1			
2			
3			

HEAP CLUSTERING

Tuples are sorted in the heap's pages using the order specified by a clustering index.

If the query accesses tuples using the clustering index's attributes, then the DBMS can jump directly to the pages that it needs.



INDEX SCAN

The DBMS picks an index to find the tuples that the query needs.

Lecture 14

Which index to use depends on:

- What attributes the index contains
- What attributes the query references
- The attribute's value domains
- Predicate composition
- Whether the index has unique or non-unique keys

INDEX SCAN

Suppose that we have a single table with 100 tuples and two indexes:

- Index #1: **age**
- Index #2: **dept**

```
SELECT * FROM students
WHERE age < 30
      AND dept = 'CS'
      AND country = 'US'
```

Scenario #1

There are 99 people under the age of 30 but only 2 people in the CS department.

Scenario #2

There are 99 people in the CS department but only 2 people under the age of 30.

MULTI-INDEX SCAN

If there are multiple indexes that the DBMS can use for a query:

- Compute sets of record ids using each matching index.
- Combine these sets based on the query's predicates (union vs. intersect).
- Retrieve the records and apply any remaining predicates.

Postgres calls this Bitmap Scan.

MULTI-INDEX SCAN

With an index on **age** and an index on **dept**,

- We can retrieve the record ids satisfying **age < 30** using the first,
- Then retrieve the record ids satisfying **dept = 'CS'** using the second,
- Take their intersection
- Retrieve records and check **country = 'US'**.

```
SELECT * FROM students
WHERE age < 30
      AND dept = 'CS'
      AND country = 'US'
```

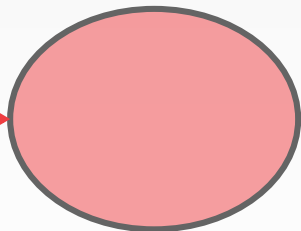
MULTI-INDEX SCAN

Set intersection can be done with
bitmaps, hash tables, or Bloom filters.



age < 30

record ids

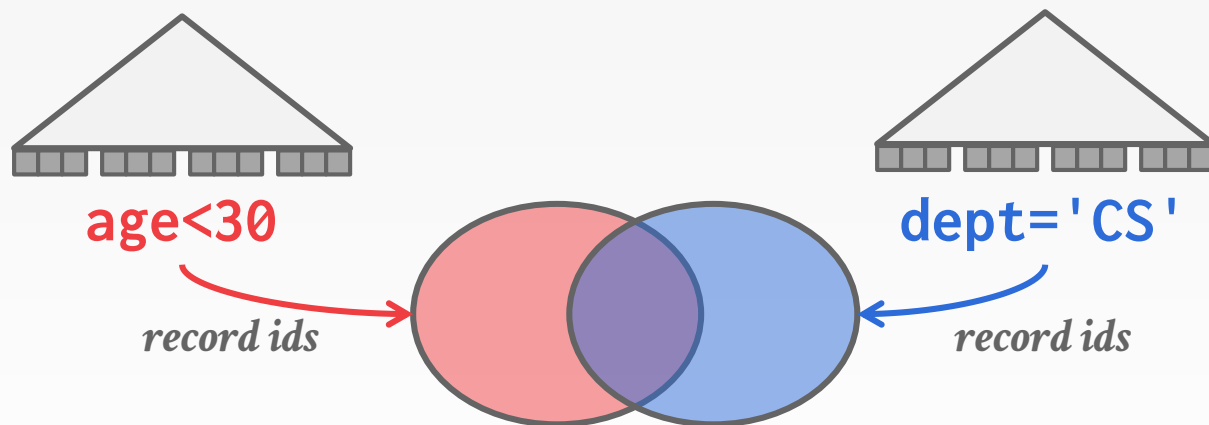


dept = 'CS'

```
SELECT * FROM students
WHERE age < 30
      AND dept = 'CS'
      AND country = 'US'
```

MULTI-INDEX SCAN

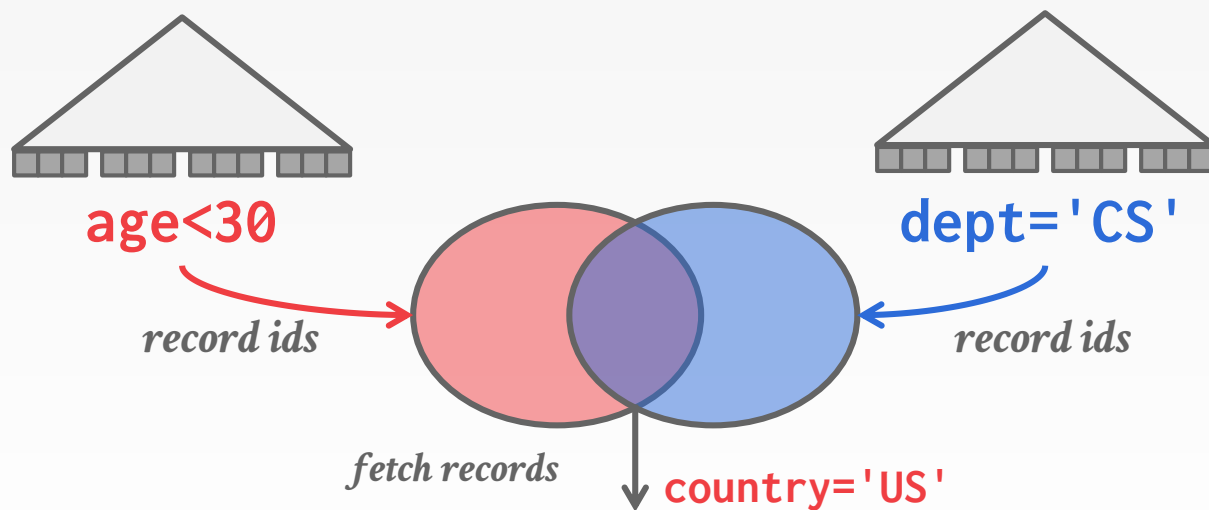
Set intersection can be done with
bitmaps, hash tables, or Bloom filters.



```
SELECT * FROM students
WHERE age < 30
      AND dept = 'CS'
      AND country = 'US'
```


MULTI-INDEX SCAN

Set intersection can be done with
bitmaps, hash tables, or Bloom filters.



```
SELECT * FROM students
WHERE age < 30
      AND dept = 'CS'
      AND country = 'US'
```

INDEX SCAN PAGE SORTING

Retrieving tuples in the order that appear in an unclustered index is inefficient.

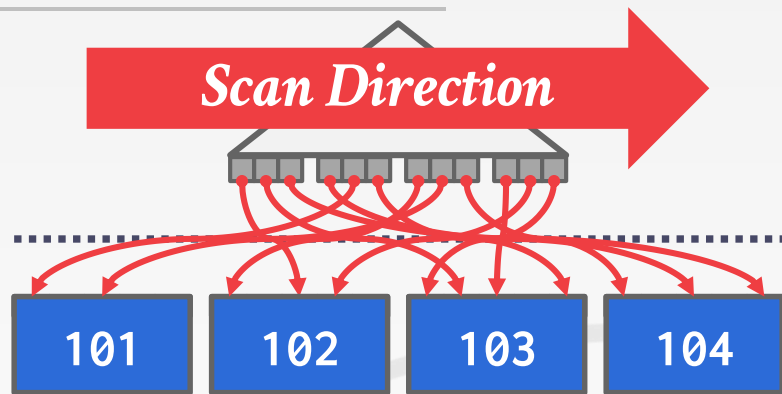
The DBMS can first figure out all the tuples that it needs and then sort them based on their page id.



INDEX SCAN PAGE SORTING

Retrieving tuples in the order that appear in an unclustered index is inefficient.

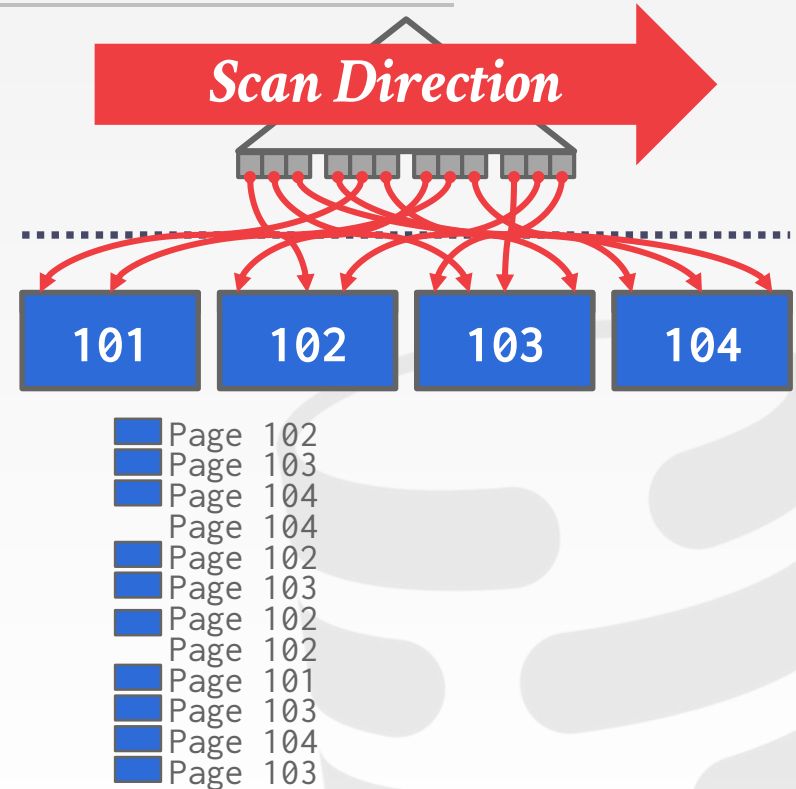
The DBMS can first figure out all the tuples that it needs and then sort them based on their page id.



INDEX SCAN PAGE SORTING

Retrieving tuples in the order that appear in an unclustered index is inefficient.

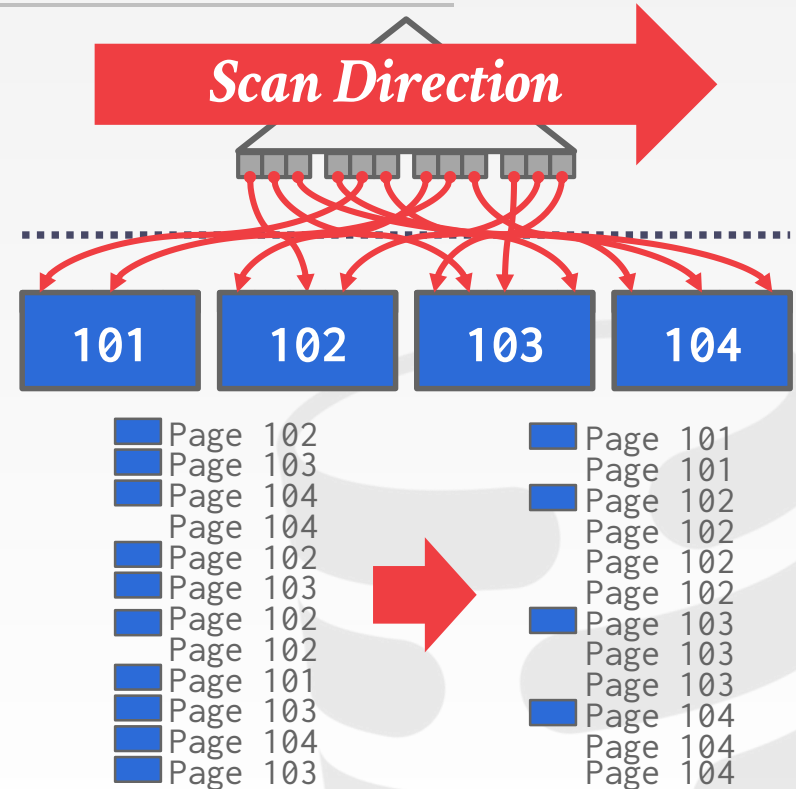
The DBMS can first figure out all the tuples that it needs and then sort them based on their page id.



INDEX SCAN PAGE SORTING

Retrieving tuples in the order that appear in an unclustered index is inefficient.

The DBMS can first figure out all the tuples that it needs and then sort them based on their page id.



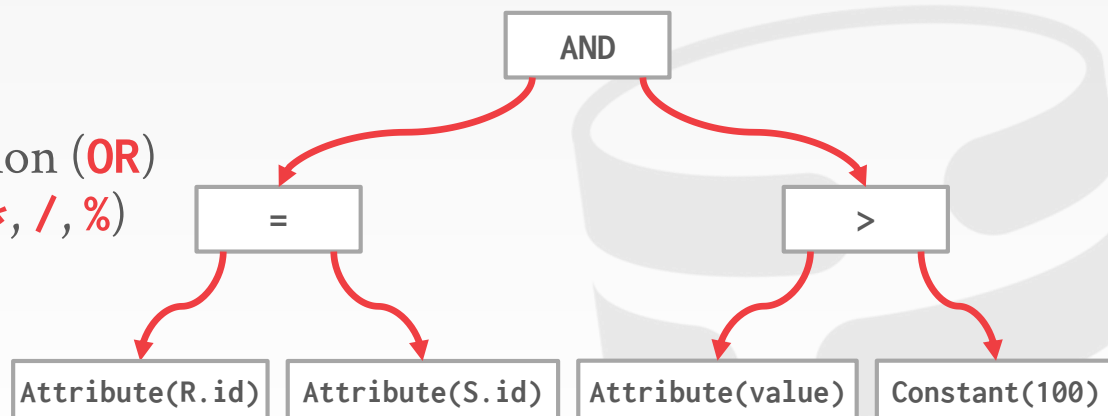
EXPRESSION EVALUATION

The DBMS represents a **WHERE** clause as an expression tree.

The nodes in the tree represent different expression types:

- Comparisons (**=**, **<**, **>**, **!=**)
- Conjunction (**AND**), Disjunction (**OR**)
- Arithmetic Operators (**+**, **-**, *****, **/**, **%**)
- Constant Values
- Tuple Attribute References

```
SELECT R.id, S.cdate
FROM R JOIN S
  ON R.id = S.id
WHERE S.value > 100
```



EXPRESSION EVALUATION

```
SELECT * FROM S  
WHERE B.value = ? + 1
```

EXPRESSION EVALUATION

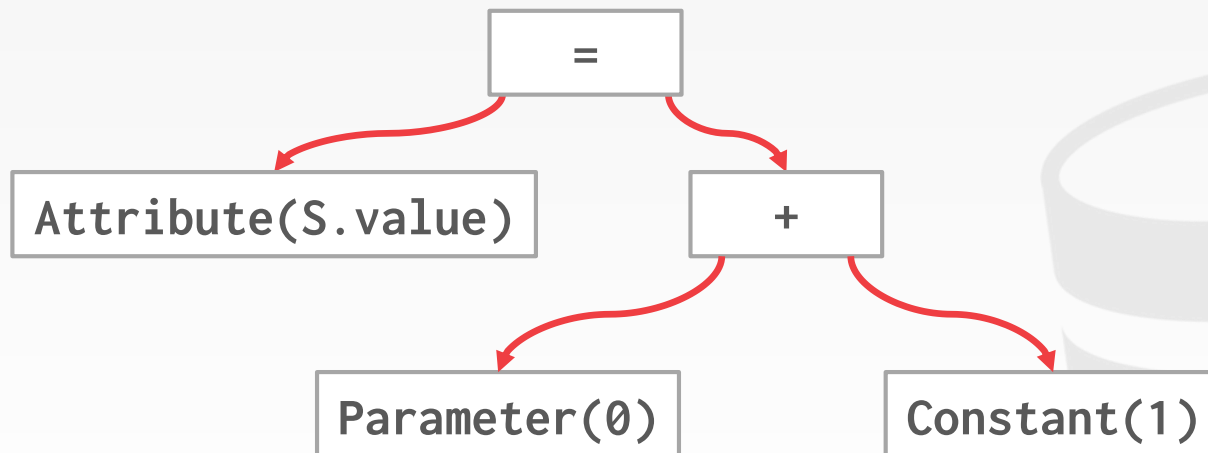
Execution Context

```
SELECT * FROM S  
WHERE B.value = ? + 1
```

Current Tuple
(123, 1000)

Query Parameters
(int:999)

Table Schema
S→(int:id, int:value)



EXPRESSION EVALUATION

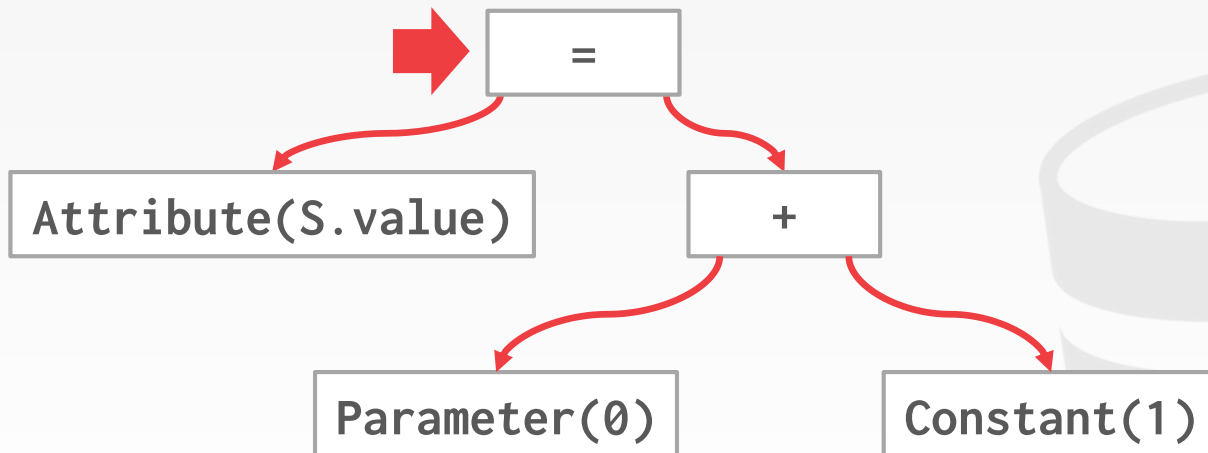
Execution Context

```
SELECT * FROM S  
WHERE B.value = ? + 1
```

Current Tuple
(123, 1000)

Query Parameters
(int:999)

Table Schema
S→(int:id, int:value)



EXPRESSION EVALUATION

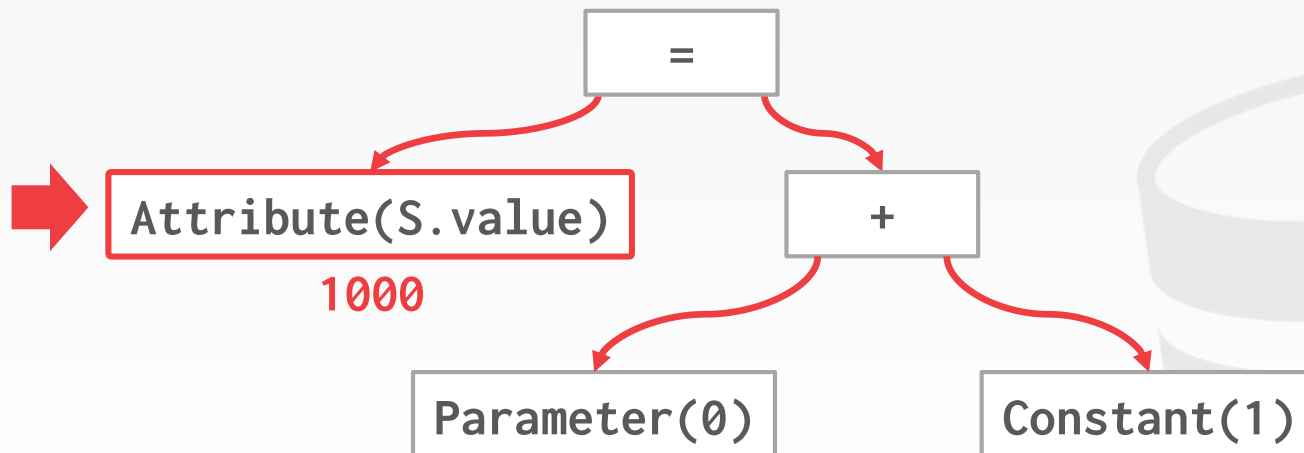
Execution Context

```
SELECT * FROM S  
WHERE B.value = ? + 1
```

Current Tuple
(123, 1000)

Query Parameters
(int:999)

Table Schema
S→(int:id, int:value)



EXPRESSION EVALUATION

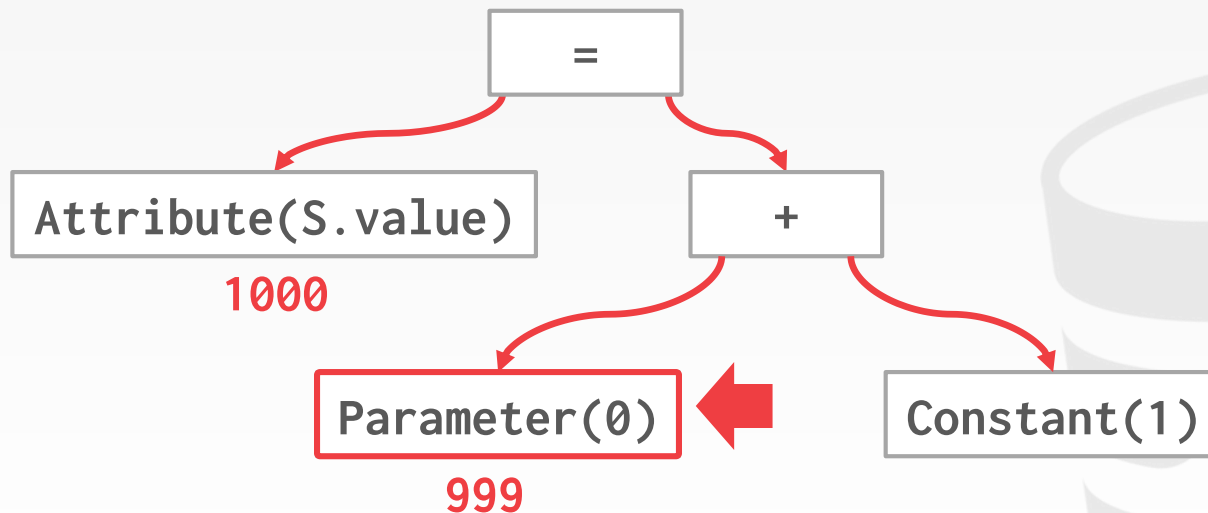
Execution Context

```
SELECT * FROM S  
WHERE B.value = ? + 1
```

Current Tuple
(123, 1000)

Query Parameters
(int:999)

Table Schema
S→(int:id, int:value)



EXPRESSION EVALUATION

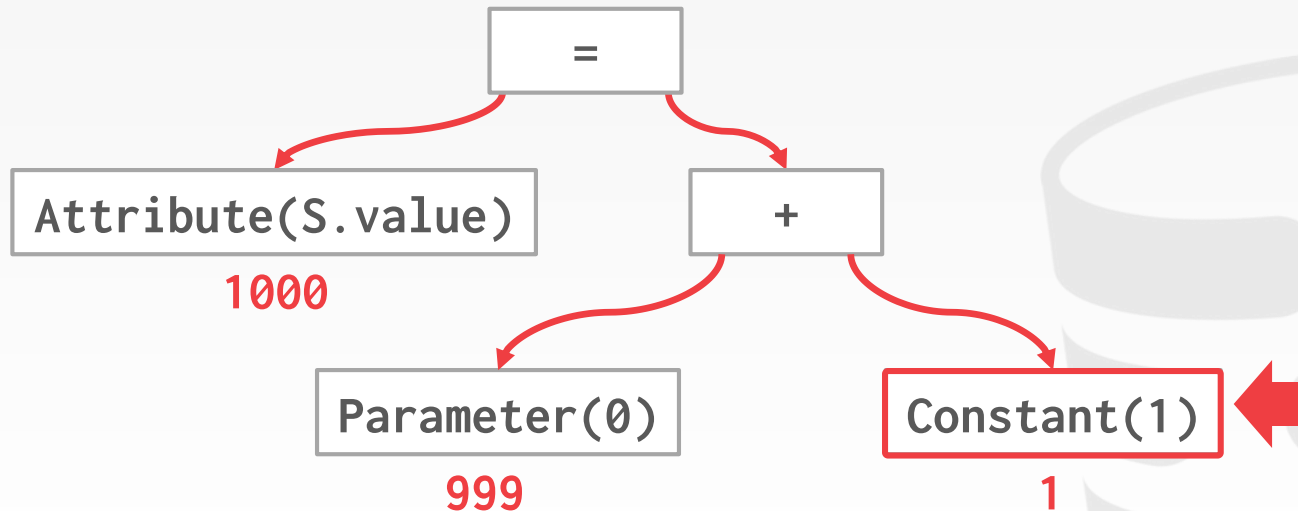
Execution Context

```
SELECT * FROM S  
WHERE B.value = ? + 1
```

Current Tuple
(123, 1000)

Query Parameters
(int:999)

Table Schema
S→(int:id, int:value)



EXPRESSION EVALUATION

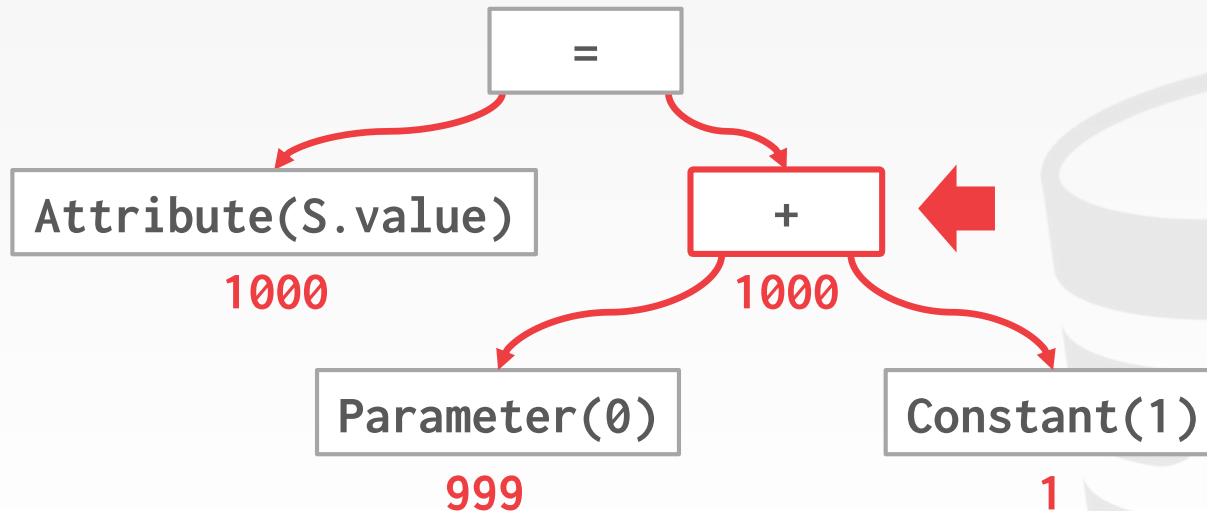
Execution Context

```
SELECT * FROM S  
WHERE B.value = ? + 1
```

Current Tuple
(123, 1000)

Query Parameters
(int:999)

Table Schema
S→(int:id, int:value)



EXPRESSION EVALUATION

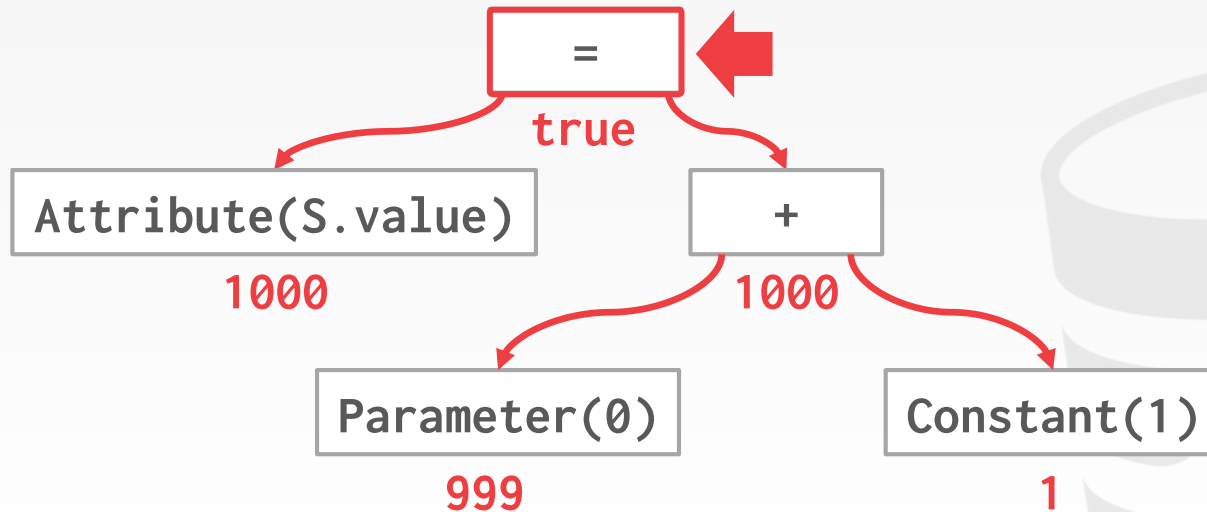
Execution Context

SELECT * FROM S
WHERE B.value = ? + 1

Current Tuple
(123, 1000)

Query Parameters
(int:999)

Table Schema
S→(int:id, int:value)



EXPRESSION EVALUATION

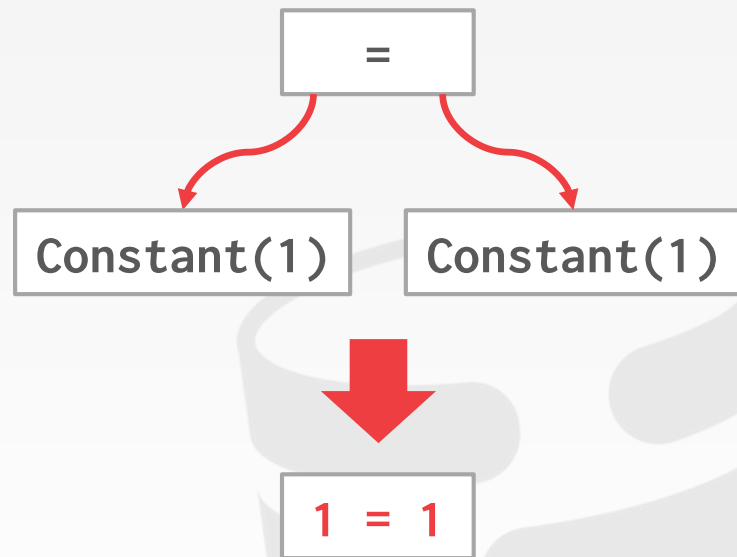
Evaluating predicates in this manner is slow.

→ The DBMS traverses the tree and for each node that it visits it must figure out what the operator needs to do.

Consider the predicate "**WHERE 1=1**"

A better approach is to just evaluate the expression directly.

→ Think JIT compilation



CONCLUSION

The same query plan can be executed in multiple ways.

(Most) DBMSs will want to use an index scan as much as possible.

Expression trees are flexible but slow.



NEXT CLASS

Parallel Query Execution



CMU 14-455 (2019) · 课程资料包 @ShowMeAI



视频

中英双语字幕



课件

一键打包下载



笔记

官方笔记翻译



代码

作业项目解析



视频 · B 站 [扫码或点击链接]

<https://www.bilibili.com/video/BV1qf4y1J7mX>



课件 & 代码 · 博客 [扫码或点击链接]

<http://blog.showmeai.tech/cmu-14-455/>

高级 SQL
数据库

聚合
内存管理
树索引

排序
查询规划
索引构建

并发控制
查询执行

Awesome AI Courses Notes Cheatsheets 是 [ShowMeAI](#) 资料库的分支系列，覆盖最具知名度的 **TOP50+** 门 AI 课程，旨在为读者和学习者提供一整套高品质中文学习笔记和速查表。

点击课程名称，跳转至课程**资料包**页面，**一键下载**课程全部资料！

机器学习	深度学习	自然语言处理	计算机视觉
Stanford · CS229	Stanford · CS230	Stanford · CS224n	Stanford · CS231n
# Awesome AI Courses Notes Cheatsheets · 持续更新中			
知识图谱	图机器学习	深度强化学习	自动驾驶
Stanford · CS520	Stanford · CS224W	UCBerkeley · CS285	MIT · 6.S094



微信公众号

资料下载方式 2：扫码点击**底部菜单栏**
称为 **AI 内容创作者**？回复 [添砖加瓦]